

CSC4120 Design and Analysis of Algorithms

L0 Introduction

Main Course Topics

- Key algorithm concepts (asymptotic complexity, divide-and-conquer)
- Basic data structure algorithms
 - Heaps (priority queues)
 - Binary search trees (scheduling)
 - Hashing (cryptography, dictionaries)
- Graph searches (reachability analysis)
- Shortest paths (Google maps, navigation)
- Greedy algorithms (minimum spanning trees, Huffman codes)
- Dynamic programming (any optimization problem)
- Network flows
- Complexity (NP-complete, poly reductions, approximation algorithms)

Textbook: **Algorithms** by S. Dasgupta, C.H. Papadimitriou, and U.V. Vazirani.

L1 Basic Concepts of Algorithmic Performance

Asymptotic Complexity

What is $T(n)$?

$T(n)$ is the largest (worst) execution time over all inputs of size n .

Asymptotic Complexity

Goal: Capture the growth of $T(n)$ of an algorithm as the problem instance size $n \rightarrow \infty$

Key Idea: Keep only the rate of growth, i.e., the **dominant term when $n \rightarrow \infty$** , forgetting multiplicative constants and lower order terms.

Three Os

1. Θ : Grows asymptotically as fast as ($=$)

- Tight Bound
- $f(n) = \Theta(g(n)) \Leftrightarrow \exists c_1, c_2 > 0, n_0 \text{ s.t. } c_1 g(n) \leq f(n) \leq c_2 g(n) \text{ for } n \geq n_0$
- e.g. $100n^2 + 5n - 5 = 0.1n^2 + 10n^{1.5} - 5 = \Theta(n^2)$

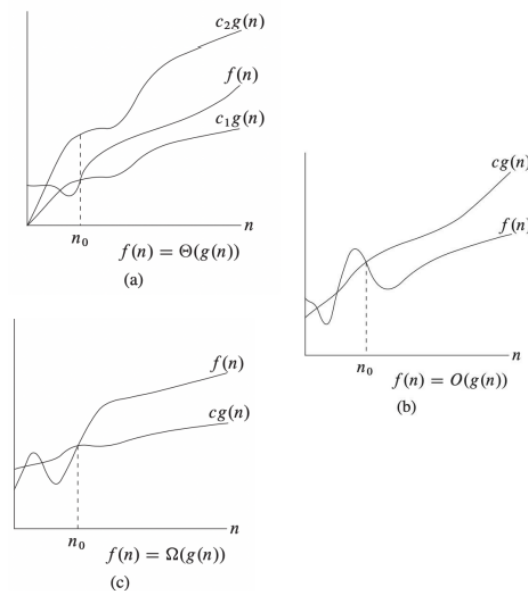
2. O : Grows asymptotically *at most* as fast as ($\leq$)

- Upper Bound
- $f(n) = O(g(n)) \Leftrightarrow \exists c > 0, n_0 \text{ s.t. } f(n) \leq c g(n) \text{ for } n \geq n_0$

- e.g. $2n^{1.5} - 5 \leq n^{1.5} \leq n^2 \Rightarrow 2n^{1.5} - 5 = O(n^{1.5}), = O(n^2)$
- e.g. $n^{1.5} \leq n^2 \leq 2^n \Rightarrow n^{1.5} = O(n^2), n^2 = O(2^n), n^{1.5} = O(2^n)$

3. Ω : Grows asymptotically *at least* as fast as ($\geq$)

- Lower Bound
- $f(n) = \Omega(g(n)) \Leftrightarrow \exists c > 0, n_0 \text{ s.t. } f(n) \geq c g(n) \text{ for } n \geq n_0$
- e.g. $5 \leq 2n \leq n^{1.5} + 1 \leq 2^n \Rightarrow 2n = \Omega(1), 2^n = \Omega(n^{1.5})$



In Practice

Data Size	"Practical" Algorithm					Practically ineffective algorithm
	n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3
step = 1 μ s						
10	3.3 μ s	10 μ s	33 μ s	100 μ s	1 ms	1 ms
10^2	6.6 μ s	100 μ s	664 μ s	10 ms	1 s	4×10^{16} years
10^4	13 μ s	10 ms	133 ms	1.7 minutes	11.6 days	10^{2997} years
10^6	20 μ s	1 s	20 s	11.6 days	32000 years	10^{300000} years

The time needed to solve a problem

Conclusions

- To measure the performance of algorithms we measure their asymptotic complexity.
- Big O is used to describe an **upper bound** for the running time of an algorithm.
- Big Ω is used to describe a **lower bound** for the running time of an algorithm.
- Big Θ is used to denote **asymptotic equivalence** of the running times of two algorithms. And when $\Omega(g(n)) = O(g(n)) = f(n)$, we can have $\Theta(g(n)) = f(n)$

L2 Divide and Conquer

Divide and Conquer (Sorting and Master Theorem)

Basic Devide and Conquer

There are 3 steps:

1. *Divide* input into multiple **smaller** (usually disjoint) points.
2. *Conquer* each of the parts separately (using recursive calls).
3. *Combine* results from different calls.

$$T(n) = \text{divide time} + T(n_1) + T(n_2) + \dots + T(n_k) + \text{combine time}$$

Sorting

A non-D&C solution: **Insertion Sort**

```
void insertionSort(int arr[], int n) {
    for (int i = 1; i < n; i++) {
        int key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = key;
    }
}
```

We have the complexity is $O(n^2)$ since it runs two loops with n steps.

But $O(n^2)$ is too slow for most of the problem, can it be much faster? Of course. We can use divide and conquer here!

A D&C solution: **Merge Sort**

```
void merge(vector<int>& arr, int l, int m, int r) {
    int n1 = m - l + 1;
    int n2 = r - m;
    vector<int> L(n1), R(n2);
    for (int i = 0; i < n1; i++) L[i] = arr[l + i];
    for (int j = 0; j < n2; j++) R[j] = arr[m + 1 + j];
    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) arr[k++] = L[i++];
        else arr[k++] = R[j++];
    }
    while (i < n1) arr[k++] = L[i++];
    while (j < n2) arr[k++] = R[j++];
}
```

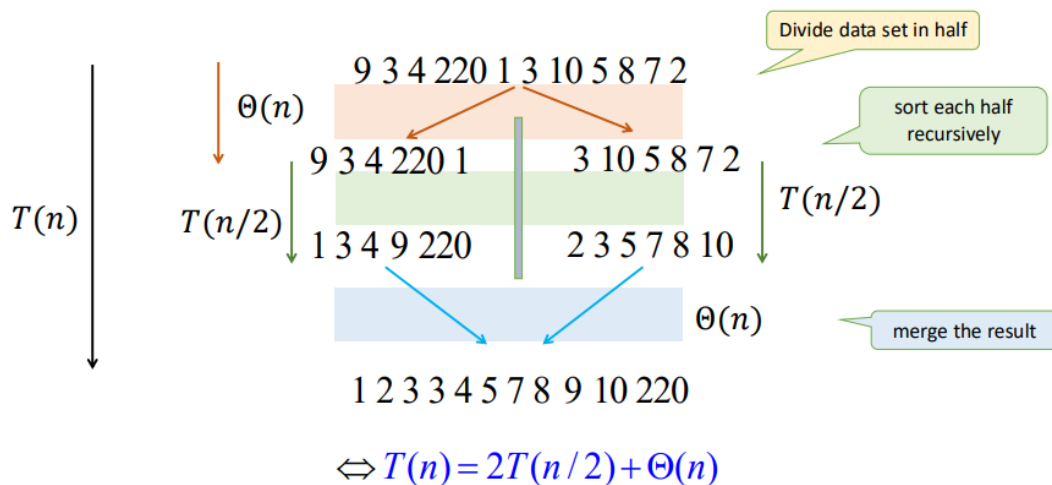
```

}

void mergeSort(vector<int>& arr, int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(arr, l, m);
        mergeSort(arr, m + 1, r);
        merge(arr, l, m, r);
    }
}

```

From the function `mergeSort` we can see that we divide the array into 2 parts and conquer them separately and recursively. And the `merge` function has the time complexity of $\Theta(n)$.



From the graph we can see that there is only $\log n$ levels of recursion. Thus, we have

$$T(n) = 2T(n/2) + cn \quad \&= (cn) \log n + nT(1) \quad \&= \Theta(n \log n)$$

Where cn denotes for the time of divide and merge, and $T(1)$ represents for the node of the recursion tree.

Geometric Sums

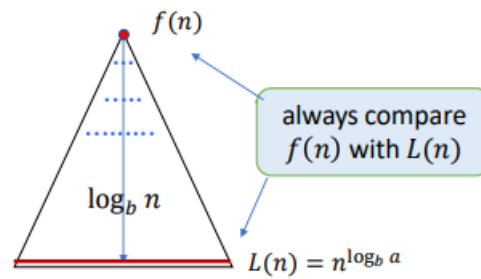
Consider a geometric series $S = 1 + x + x^2 + \dots + x^h = \frac{x^{h+1} - 1}{x - 1}$ (for $x \neq 1$). If $x < 1$, then $1 + x + x^2 + \dots + x^h \in \left[1, \frac{1}{1-x} \right] \Rightarrow S = \Theta(1)$. If $x > 1$, then $1 + x + x^2 + \dots + x^h < \frac{1}{1-x} x^{h+1} \Rightarrow S = \Theta(x^h)$.

Master Theorem

Consider the recursion:

$$T(n) = aT(n/b) + f(n)$$

- Number of levels: $\log_b n = \Theta(\log n)$
- Number of leaves: $L = a^{\log_b n} = n^{\log_b a}$



We use the recursion tree to calculate the cost. The cost of the i -th (root $i=0$) is:
$$C_i = (\text{Nodes Number of } i \text{th Level}) \times (\text{cost of Every Subproblem}) = a^i f(n/b^i)$$
 So that, we have:
$$T(n) = \sum_{i=0}^{\log_b n} C_i = \sum_{i=0}^{\log_b n} a^i f(n/b^i)$$

We need to compare the time within root and leaves to get the main factor. The usual method is to compare $f(n)$ and $L(n) = n^{\log_b a}$

There are 3 situations here:

1. Geometrically Increasing (**Leaves**): $f(n) = O(L^{1-\epsilon}) \Rightarrow T(n) = \Theta(L)$ e.g. $T(n) = 2T(n/2) + \sqrt{n}$, we have $a=2, b=2, L=n^{\log_2 2} = n$, and $\sqrt{n} = n^{1/2} = O(L^{1-\epsilon})$, thus $T(n) = \Theta(n)$
2. Roughly **Equal** Levels: $f(n) = \Theta(L) \Rightarrow T(n) = \Theta(L \log n)$ e.g. $T(n) = 2T(n/2) + n$, we have $L=n^{\log_2 2} = n$, and $n = O(L)$, thus $T(n) = \Theta(n \log n)$
3. Geometrically Decreasing (**Root**) + Regularly Condition $f(n) = \Omega(L^{1+\epsilon})$ and $af(n/b) \leq sf(n), s < 1 \Rightarrow T(n) = \Theta(f(n))$ e.g. $T(n) = 2T(n/2) + n^2$, we have $L=n^{\log_2 2} = n$, and $n^2 > O(L)$, $2f(n/2) = 2(n/2)^2 \leq (1/2)f(n)$, thus $T(n) = \Theta(n^2)$

Applications of D&C

Convex Hull

Given n points $S = \{(x_i, y_i) \mid i = 1, 2, \dots, n\}$ s.t. all have different x and y coordinates, and no three of them in a line.

Convex Hull (CH): The smallest polygon containing all points in S . Represented by the sequence of points in the boundary in clockwise order.

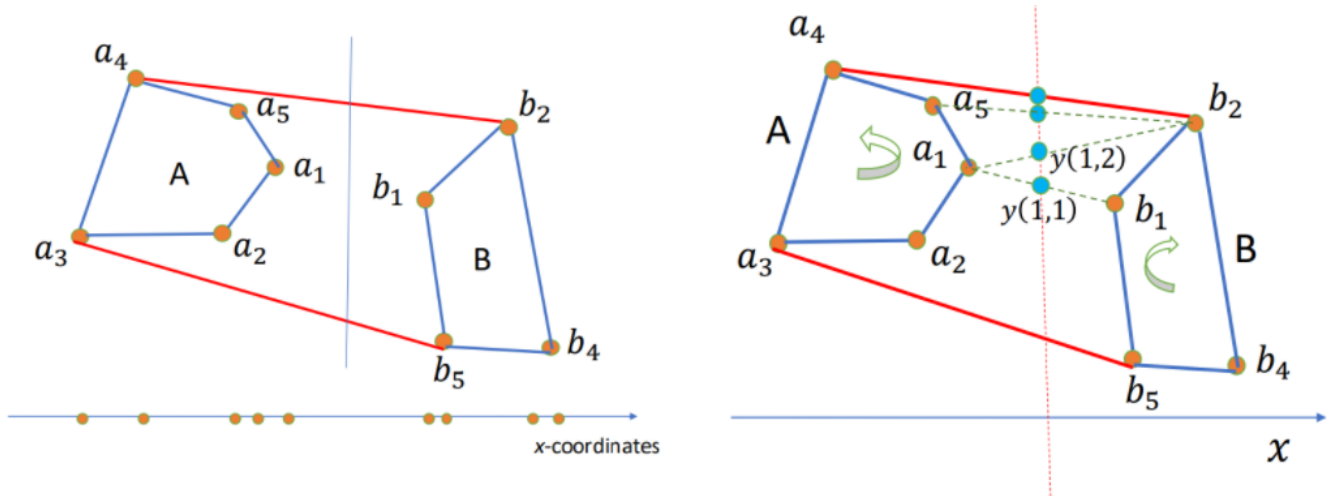
How to use D&C here?

1. Sort points by x -coordinate.
2. For our input set S of points:
 - Divide (by x -coordinates) into left-half A and right-half B .
 - Compute $CH(A)$ and $CH(B)$.
 - Merge $CH(A)$ and $CH(B)$ to get $CH(A+B)$.

It has the complexity time $T(n) = 2T(n/2) + \Theta(n) \Rightarrow \Theta(n \log n)$.

BUT, how to merge?

Algorithm: Start with (a_1, b_1) . While $y(i, j)$ increases, keep moving by one step either a_i counterclockwise or b_j clockwise. When cannot increase anymore, we found the upper tangent, say a_u, b_u . Similarly, the lower tangent a_l, b_l . Then the CH = points of A clockwise from a_l to a_u + points of B clockwise from b_u to b_l . Hence total time $\Theta(n)$



Median Finding / Peak Finding

Not interested.

Fast Fourier Transform

Multiplication of polynomials as *list of coefficients*.

Consider:
$$\begin{aligned} A(x) &= a_0 + a_1 x + \dots + a_n x^n = \sum_{i=0}^n a_i x^i \\ B(x) &= b_0 + b_1 x + \dots + b_n x^n = \sum_{i=0}^n b_i x^i \end{aligned}$$
 To calculate:
$$\begin{aligned} C(x) &= A(x) B(x) \\ C(x) &= c_0 + c_1 x + \dots + c_{2n} x^{2n} = \sum_{i=0}^{2n} c_i x^i \end{aligned}$$
 where $c_k = a_0 b_k + a_1 b_{k-1} + \dots + a_k b_0 = \sum_{i=0}^k a_i b_{k-i}$

Complexity: For $c_k \rightarrow O(k)$, for $C(x) \rightarrow O(n^2)$

Can we do it **faster**? Like $O(n \log n)$?

Multiplication of polynomials as *list of values*.

Consider: A degree- n polynomial is **uniquely characterized** by its values at **any** $n+1$ distinct points $x_0, x_1, \dots, x_n$.

That is, if we fix any distinct $n+1$ points, we can specify any degree- n polynomial $A(x) = a_0 + a_1 x + \dots + a_n x^n$ by the $n+1$ values: $A(x_0), A(x_1), \dots, A(x_n)$. Then we can have:
$$\begin{aligned} C(x) &= A(x) B(x) \\ C(x_i) &= A(x_i) B(x_i) \end{aligned}$$
 where $i = 0, 1, \dots, 2n$. Polynomial multiplication is $O(n)$ using this representation!

But how can we get the coefficients of $C(x)$?

Evaluation and Interpolation

Evaluation: $a_0, a_1, \dots, a_n \rightarrow A(x_0), A(x_1), \dots, A(x_n)$

$$A(x) = M_n(x) \cdot a \quad \begin{bmatrix} A(x_0) & A(x_1) & \dots & A(x_n) \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix} \cdot \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix}$$

Interpolation: $A(x_0), A(x_1), \dots, A(x_n) \rightarrow a_0, a_1, \dots, a_n$

$$a = M_n^{-1}(x) \cdot A(x) \quad \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_n \end{bmatrix} = \begin{bmatrix} 1 & x_0 & x_0^2 & \dots & x_0^n \\ 1 & x_1 & x_1^2 & \dots & x_1^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & x_n^2 & \dots & x_n^n \end{bmatrix}^{-1} \cdot \begin{bmatrix} A(x_0) \\ A(x_1) \\ \vdots \\ A(x_n) \end{bmatrix}$$

Here M_n is Vandermonde matrix.

Consider $C(x_i) = A(x_i) B(x_i)$, from the two matrix multiplication we can know that: If we have every $A(x_i)$ and $B(x_i)$ we can use $O(n)$ to multiply and get $C(x_i)$, then transfer it into every coefficient c_i .

But, using matrix multiplication directly will need time complexity of $O(n^2)$. Can we optimize it to $O(n \log n)$? Yes, and here comes **Fast Fourier Transform (FFT)**.

FFT

Let's calculate evaluation using FFT first.

Consider a polynomial $P(x)$ with length of even number n s.t.
$$P(x) = a_0 + a_1 x + a_2 x^2 + \dots + a_{n-1} x^{n-1} = P_{\text{even}}(x^2) + x \cdot P_{\text{odd}}(x^2) \quad \text{where } P_{\text{even}}(x) = a_0 + a_2 x + a_4 x^2 + \dots + a_{n-2} x^{n/2} \quad P_{\text{odd}}(x) = a_1 + a_3 x + a_5 x^2 + \dots + a_{n-1} x^{n/2}$$

By doing this, we **divide** the the length n to two length $n/2$. Then, we have to **conquer** them each and then **combine** them together.

Let A_k denote $A(x_k)$.

The question is: How can we divide A_k ?

And here's the most essential part: We use **primitive root** ω .

Now, think about an complex equation: $z^n = (e^{i\theta})^n = 1$ We can get the solutions:
$$r = 1 \quad \theta = \frac{2k\pi}{n} \quad \text{where } k = 0, 1, \dots, n-1$$
 Thus,
$$\omega_n = e^{\frac{2\pi i}{n}} \quad \omega_n^k = e^{\frac{2\pi i}{n} k} = e^{i\theta} \quad (\omega_n^k)^n = 1$$
 To each k , we want to calculate:
$$A_k = P(\omega_n^k) = P_{\text{even}}(\omega_n^{2k}) + \omega_n^k \cdot P_{\text{odd}}(\omega_n^{2k}) = P_{\text{even}}(\omega_{n/2}^k) + \omega_n^k \cdot P_{\text{odd}}(\omega_{n/2}^k)$$
 Since $\omega_n^{2k} = e^{\frac{2\pi i}{n} 2k} = e^{\frac{2\pi i}{n/2} k} = \omega_{n/2}^k$ This is called the **Discrete Fourier Transform (DFT)** of A_k .

And we found that:
$$A_{k+n/2} = P(\omega_n^{k+n/2}) = P_{\text{even}}(\omega_{n/2}^{k+n/2}) + \omega_n^{k+n/2} \cdot P_{\text{odd}}(\omega_{n/2}^{k+n/2}) = P_{\text{even}}(\omega_{n/2}^k) \cdot \omega_{n/2}^{n/2} + \omega_n^k \cdot \omega_{n/2}^{n/2} \cdot P_{\text{odd}}(\omega_{n/2}^k) = P_{\text{even}}(\omega_{n/2}^k) - \omega_n^k \cdot P_{\text{odd}}(\omega_{n/2}^k)$$

$$\omega_n^k \cdot P_{\{\text{odd}\}}(\omega_{n/2}^k) \end{align*} \quad \text{Since } \omega_n^n = e^{\frac{2\pi i}{n}n} = e^{2\pi i} = 1 \quad \omega_n^{n/2} = e^{\frac{2\pi i}{n}(n/2)} = e^{\pi i} = -1 \quad$$

Combining A_k and $A_{k+n/2}$ together for $0 \leq k < n/2$, we can do D&C here! Since $P_{\text{even}}(\omega_{n/2}^k)$ and $P_{\text{odd}}(\omega_{n/2}^k)$ can calculate from the half DFT.

The time complexity is $T(n) = 2T(n/2) + O(n) = O(n \log n)$. We get the whole evaluation.

IFFT

To do interpolation, we need **Inverse Fast Fourier Tranfrom (IFFT)**.

Inverse FFT is easy to explain using Linear Algebra.

It's easy since: $a = M_n^{-1}(\omega) \cdot A(\omega) = \frac{1}{n} M_n(\omega^{-1}) \cdot A(\omega)$
And it's over.

Let $A(\omega) = \text{FFT}(a, \omega)$, then $a = \frac{1}{n} \text{FFT}(A, \omega^{-1})$.

The Whole Progress

1. Filled $A(x)$ and $B(x)$ to length of $n=2^k$ by 0 due to easier FFT D&C.
2. Calculate $A(\omega^k)$ and $B(\omega^k)$ for each $k=0,1,\dots,n-1$, where $\omega = e^{\frac{2\pi i}{n}}$ using FFT.
3. Calculate $C(\omega^k) = A(\omega^k) \cdot B(\omega^k)$.
4. Calculate c_i using IFFT.

L3 Randomized Algorithms

Basic Concepts of Randomized Algorithms

Randomized Algorithms: On the same input, different executions of the same algorithm may result in *different running times and different outputs*.

Type of **polynomial** randomized algorithms:

- **Monte Carlo:** Bounded number of steps, but might not find the answer.
 - Runs in *polynomial* time, $P_r(\text{answer is correct}) = p_o > \frac{1}{2}$.
 - Always able to check the answer for correctness.
 - How to use: Repeat if necessary, use majority of answers.
- **Las Vegas:** No guarantee on max number of steps, always finds answer.
 - Runs in *expected polynomial* time.
 - Not stop until finds the correct answer.

$\overline{T}(n)$: Expected time to terminate for a given input of size n .

Quicksort

Consider combining D&C and randomize

1. Pick an *pivot* x of the array.

2. Divide: Partition the array into subarrays using x .
3. Conquer: Recursively sort A , B .
4. Combine: A , x , B .

$\overline{T}(n) = \Theta(n \log n)$ for all inputs. $\Rightarrow$ Random selection of x makes no input to be worse.

Or we can say $\Theta(n \log n)$ is the **average complexity** of quicksort.

Frievald's Algorithm

Given $n \times n$ matrices A , B and C , check that $AB = C$ (yes or no).

It's very slow to use brute force algorithm since matrices multiplication is $O(n^3)$.

Consider randomization (Frievald's Algorithm):

1. Choose a random binary vector r .
2. If $(A(Br) = Cr) \bmod 2$ then output "yes" else "no".

Property: If $AB \neq C$, then $\text{Pr}[A(Br) \neq Cr] \geq \frac{1}{2}$.

That is, Do 10 tests, if all say "yes", the probability of error $< \left(\frac{1}{2}\right)^{10}$.

L4 Lower Bounds and Heaps

Comparison Model

Comparsion Sort

All the sorting algorithms we've seen so far are **comparison sort** (merge sort, insertion sort, bubble sort, etc.).

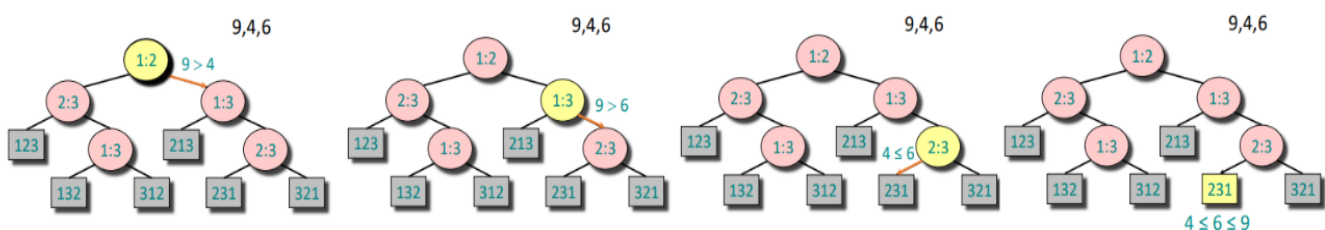
And the best running time that we've seen for comparison sort is $O(n \log n)$. *Is it the best we can do?*

Decision-Tree

A given recipe for sorting n numbers $a_1, a_2, \dots, a_n$.

Nodes are suggested comparisons: i, j means compare a_i to a_j for $i, j \in \{1, 2, \dots, n\}$

The figure below is an example of decision-tree of sorting $a_1 = 9, a_2 = 4, a_3 = 6$.



From the tree we can see that:

- A path from the root to the leaves of the tree represents a trace of comparisons that the algorithm may perform.
- The running time of the algorithm = the length of the path taken.
- **Worst-case running time = height(depth) of tree**

The tree must contain $\geq n!$ leaves since there are $n!$ possible permutations. And a height- h binary tree has $L_h \leq 2^h$ leaves.

Hence: $n! \leq L_h \leq 2^h \iff h > \log(n!) \iff h = \Omega(n \log n)$

Since the Stirling's approximation: $n! \approx \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \sqrt{\log(n!)} = n \log n - n + \frac{1}{2} \log(2\pi n)$

Theorem: Any decision tree that can sort n elements must have height $\Omega(n \log n)$.

Linear Time Sorting

As mentioned above, the comparison sorting have a lower-bound complexity of $\Omega(n \log n)$, so do linear time sorting exist?

Counting Sort

Given: An array $a_0, a_1, \dots, a_{n-1}$ of n keys to be sorted.

Count the times of number appeared between $a_{\min}$ and $a_{\max}$, then use the prefix sum array to get the final place of each element. With time complexity of $O(n+k)$, $k = a_{\max} - a_{\min} + 1$

```
void counting_sort(vector<int>& a) {
    int n = a.size();
    int mn = *min_element(a.begin(), a.end());
    int mx = *max_element(a.begin(), a.end());
    int k = mx - mn + 1;
    vector<int> count(k, 0);
    for (int x : a) count[x - mn]++;
    for (int i = 1; i < k; i++) count[i] += count[i - 1];
    vector<int> res(n);
    for (int i = n - 1; i >= 0; i--) {
        int x = a[i];
        res[--count[x - mn]] = x;
    }
    a = res;
}
```

Radix Sort

Use a **radix**(base) k for an integer x : $x = a_{d-1} k^{d-1} + \dots + a_0 k^0$ to $a_{d-1} a_{d-2} \dots a_0$

Idea: Sort on least-significant digit first with auxiliary stable sort.

We are given n integers, each integer $< M$, in base k . Then we have time complexity of $O((n+k) \log_k M)$

Heaps

Storing Objects in Data Structures

We operate on objects of the form: **object $x = (x.\text{key}, x.\text{data})$** .

The attribute $x.\text{key}$ characterizes **uniquely** the object x . And we store $x.\text{key}$ combined with a pointer to the location of $x.\text{data}$ in memory.

Inserting and searching for object x is done by inserting and searching for $x.\text{key}$. Or we can say we mostly focus on $x.\text{key}$ instead of $x.\text{data}$.

Priority Queue

S : set of elements $\{ x_1, x_2, \dots, x_n \}$.

$x_i = (\text{key}_i, \text{data}_i)$.

Operations: $\text{Insert}(S, x), \max(S), \text{Extract-max}(S), \text{Incr-key}(S, x, k)$.

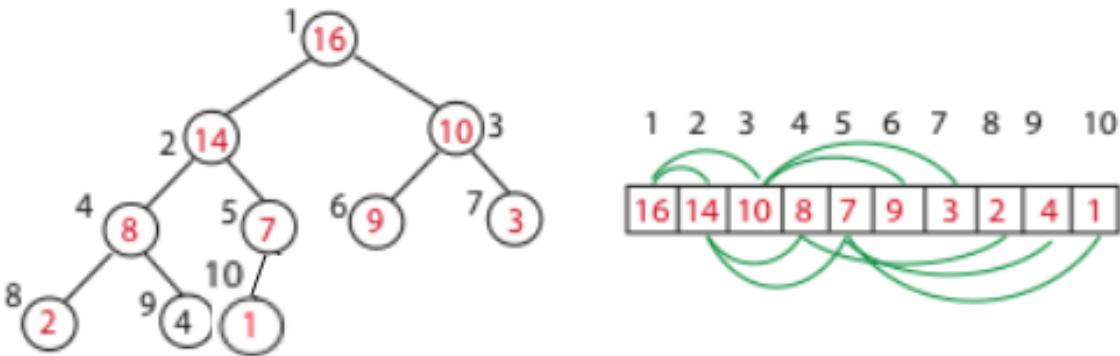
operation	Linear array	Sorted array	Heap	AVL
insert(x)	$\Theta(1)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$
min()	$\Theta(n)$	$\Theta(1)$	$\Theta(1)$	$\Theta(\log n)$
extract_min()	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$
delete(x)	$\Theta(n)$	$\Theta(n)$	$\Theta(\log n)$	$\Theta(\log n)$
find(x)	$\Theta(n)$	$\Theta(\log n)$	$\Theta(n)$	$\Theta(\log n)$

Heap

Heap: An array visualized as a nearly complete **binary tree**.

Navigate on the tree but **move on array** = full info of the tree, without pointers.

Max(Min) Heap Property: the key of a node is $\geq (\leq)$ than the keys of its children.



Binary Tree Structure in array, as shown above.

```

int s[MAXN], sz;

int parent(int i) {
    return i >> 1; // i/2
}

int left(int i) {
    return i << 1; // i*2
}

int right(int i) {
    return i << 1 | 1; // i*2+1
}

```

And the **height** of the tree is $h = \log n$.

Max Heapify: Correct a single violation of the heap property occurring at the root i of a subtree in $O(\log n)$.

```

void max_heapify(int i) {
    int l = left(i), r = right(i), maxi = i;
    if(l <= sz && s[l] > s[maxi]) maxi = l;
    if(r <= sz && s[r] > s[maxi]) maxi = r;
    if(maxi != i) {
        swap(s[i], s[maxi]);
        max_heapify(maxi);
    }
}

```

Build Max Heap: Convert an array A into a max heap. And we find that elements from $A_{\lfloor n/2 \rfloor + 1}$ to A_n are leaves of the tree, and they won't violate the rule. So we only need to heapify A_1 to $A_{\lfloor n/2 \rfloor}$. The time complexity is $O(n \log n)$.

```

void build_max_heap(int n) {
    sz = n;
    for(int i = n/2; i >= 1; i--) max_heapify(i);
}

```

Insert: We first put it at the back of the array, then go up in the tree one level by one level until no violation.

```

void insert(int val) {
    sz++;
    s[sz] = val;
    int i = sz;
    while(i > 1 && s[parent(i)] < s[i]) {
        swap(s[i], s[parent(i)]);
    }
}

```

```

        i = parent(i);
    }
}

```

Extract Max: The max element must be the first one of the array, we can just swap it with the last element, then heapify.

```

int extract_max() {
    if(!sz) return -1;
    int maxn = s[1];
    s[1] = s[sz];
    sz--;
    max_heapify(1);
    return maxn;
}

```

Heap Sort: Like extracting max element for n times. Time complexity $O(n \log n)$.

```

void heap_sort() {
    int n = sz;
    for(int i = n; i >= 2; i--) {
        swap(s[1], s[i]);
        sz--;
        max_heapify(1);
    }
}

```

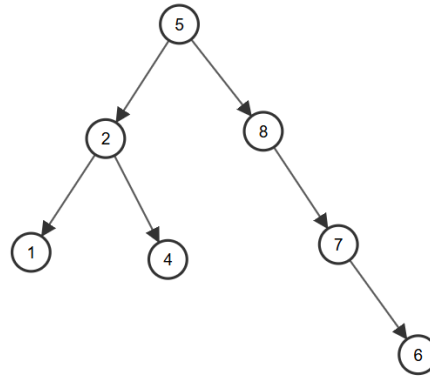
L5 Trees

Binary Search Tree

A **Binary Search Tree(BST)** is a binary tree structure that every node x of it must satisfies:

1. All key from its left subtree is **less** than $x.\text{key}$
2. All key from its right subtree is **greater** than $x.\text{key}$

And we can easily know that the left subtree and right subtree of x are both BST.



Define Node: Using C++ `struct` and pointers.

```

struct node {
    int key;
    node* left;
    node* right;
    node(int k): key(k), left(NULL), right(NULL) {}
};
  
```

Insert: Start from the root, if the key is less, put it to the left, else to the right. Doing the whole progress in recursion, until find an empty place and put it in. With average time complexity $O(\log n)$, but **worst** $O(n)$.

```

node* insert(node* root, int key) {
    if(!root) return new node(key);
    if(key < root->key) {
        root->left = insert(root->left, key);
    } else if(key > root->key) {
        root->right = insert(root->right, key);
    }
    return root;
}
  
```

Build Tree: Actually there's no such a thing, you just need to insert the new node into root.

```

node* root = NULL;
root = insert(root, a);
  
```

Inorder Traversal: The inorder traversal of the BST is a sorted array, due to the property of it.

```

void printTree(node* root) {
    if(!root) return;
    printTree(root->left);
    cout << root->key << " ";
}
  
```

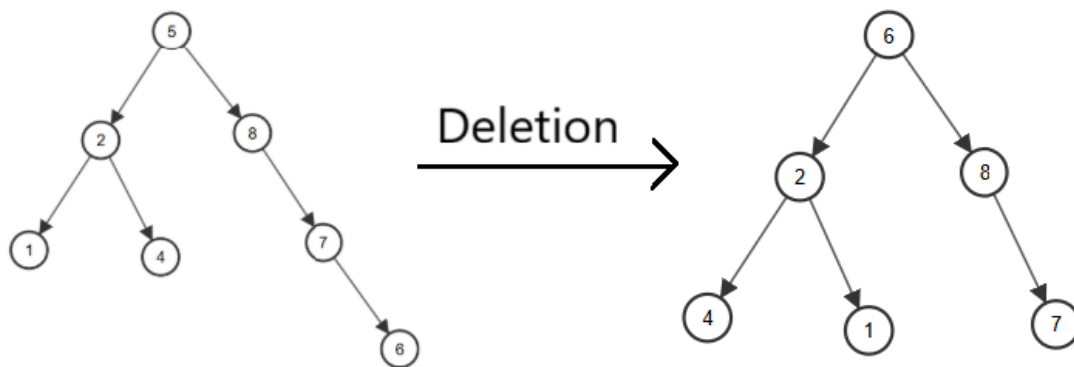
```
    printTree(root->right);
}
```

Search: Typical binary search on tree.

```
node* search(node* root, int key) {
    if(!root || root->key == key) {
        return root;
    }
    if(key < root->key) {
        return search(root->left, key);
    } else {
        return search(root->right, key);
    }
}
```

Delete: A bit complicated.

1. Leaf (no children): Delete directly.
2. On child: Use the child to replace it.
3. Two children: Find the **minimum** of **right** subtree, replace it with the element we want to delete, and delete it.



```
node* deleteNode(node* root, int key) {
    if(!root) return NULL;

    if(key < root->key)
        root->left = deleteNode(root->left, key);
    else if(key > root->key)
        root->right = deleteNode(root->right, key);
    else {
        if(!root->left && !root->right) {
            delete root;
            return NULL;
        } else if(!root->left) {
            node* temp = root->right;
            delete root;
            return temp;
        } else if(!root->right) {
            node* temp = root->left;
            delete root;
            return temp;
        }
        // Two children: Find the minimum of right subtree
        node* temp = root->right;
        while(temp->left) temp = temp->left;
        root->right = deleteNode(temp->right, temp->key);
        temp->left = root->left;
        delete root;
        return temp;
    }
}
```

```

    return temp;
} else if(!root->right) {
    node* temp = root->left;
    delete root;
    return temp;
} else {
    node* temp = findMin(root->right);
    root->key = temp->key;
    root->right = deleteNode(root->right, temp->key);
}
}
return root;
}

```

Warning: We usually use the different key to insert, but if there exists two same keys, we can do a **count**, or change the right subtree to **greater or equal** subtree.

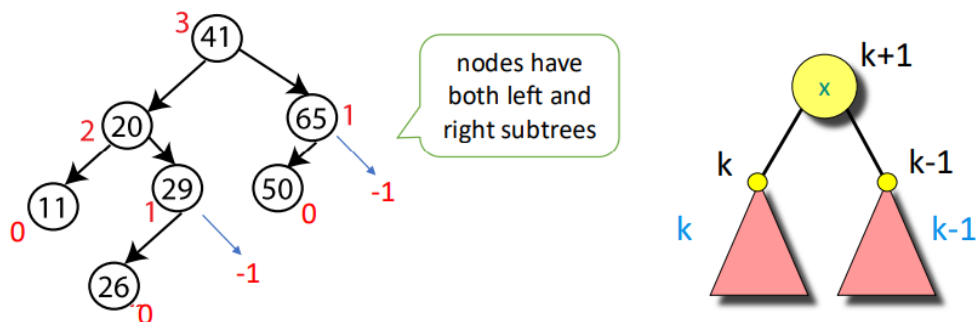
AVL Tree

An **AVL** (**Adelson-Velskii** and **Landis**) **Tree** is a **self-balancing binary search tree**.

Property: For every node, store its **height**:

- Leaves have height 0
- NIL has height -1

Invariant: For every node x , the heights of its **left child** and **right child** differ by at most 1 .



And we can know that AVL trees always have height $\Theta(\log n)$.

Define Node: Like BST, but with height.

```

struct node {
    int key, height;
    node* left;
    node* right;
    node(int k): key(k), height(1), left(NULL), right(NULL) {}
};

```

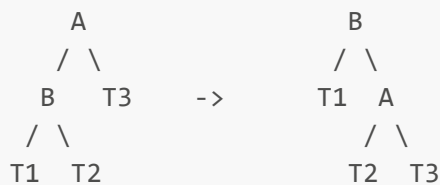

Get Height: For convenience, I take leaves height as \$1\$, NIL height as \$0\$.

```
int getHeight(node* u) {
    return u ? u->height : 0;
}
```

Get Balance: Use the definition of balance, $\text{BL} = \text{height}(\text{left subtree}) - \text{height}(\text{right subtree})$, notice that if $\text{BL} > 1$, left subtree is heavy, else if $\text{BL} < -1$, right subtree is heavy, else it's balanced at this node.

```
int getBalance(node* u) {
    return u ? getHeight(u->left) - getHeight(u->right) : 0;
}
```

Right Rotate:



```
node* rightRotate(node* u) {
    node* v = u->left;
    node* w = v->right;
    v->right = u;
    u->left = w;
    u->height = max(getHeight(u->left), getHeight(u->right)) + 1;
    v->height = max(getHeight(v->left), getHeight(v->right)) + 1;
    return v;
}
```

Left Rotate:



```

node* leftRotate(node* u) {
    node* v = u->right;
    node* w = v->left;
    v->left = u;
    u->right = w;
    u->height = max(getHeight(u->left), getHeight(u->right)) + 1;
    v->height = max(getHeight(v->left), getHeight(v->right)) + 1;
    return v;
}

```

Insert: Consider insertion with 4 situations (L and R means insert into left and right subtree):

1. LL: Do right rotation once.
2. RR: Do left rotation once.
3. LR: Do left rotation, then right rotation.
4. RL: Do right rotation, then left rotation.

```

node *insert(node* u, int key) {
    if(!u) return new node(key);
    if(key < u->key) u->left = insert(u->left, key);
    else if(key > u->key) u->right = insert(u->right, key);
    else return u;
    u->height = max(getHeight(u->left), getHeight(u->right)) + 1;
    int BL = getBalance(u);
    if(BL > 1 && key < u->left->key) return rightRotate(u); // LL
    if(BL < -1 && key > u->right->key) return leftRotate(u); // RR
    if(BL > 1 && key > u->left->key) { // LR
        u->left = leftRotate(u->left);
        return rightRotate(u);
    }
    if(BL < -1 && key < u->right->key) { // RL
        u->right = rightRotate(u->right);
        return leftRotate(u);
    }
    return u;
}

```



Delete: As same as deletion in BST, then check whether it's balanced.

```

node* findMin(node* u) {
    while(u->left) u = u->left;
    return u;
}

node* deleteNode(node* u, int key) {
    // BST Deletion first
}

```

```

    if(!u) return NULL;
    if(key < u->key) u->left = deleteNode(u->left, key);
    else if(key > u->key) u->right = deleteNode(u->right, key);
    else {
        if(!u->left || !u->right) {
            node* temp = u->left ? u->left : u->right;
            if(!temp) {
                temp = u;
                u = NULL;
            } else {
                *u = *temp;
            }
            delete temp;
        } else {
            node* temp = findMin(u->right);
            u->key = temp->key;
            u->right = deleteNode(u->right, temp->key);
        }
    }
    if(!u) return NULL;
    // Keep balance
    u->height = max(getHeight(u->left), getHeight(u->right)) + 1;
    int BL = getBalance(u);
    if(BL > 1 && getBalance(u->left) >= 0)
        return rightRotate(u); // LL
    if(BL > 1 && getBalance(u->left) < 0) {
        u->left = leftRotate(u->left);
        return rightRotate(u); // LR
    }
    if(BL < -1 && getBalance(u->right) <= 0)
        return leftRotate(u); // RR
    if(BL < -1 && getBalance(u->right) > 0) {
        u->right = rightRotate(u->right);
        return leftRotate(u); // RL
    }
    return u;
}

```

Time Complexity: In balanced BST, all operations take $O(\log n)$.

Red-Black Tree

Shit.

Interval Tree

Interval Tree is a balanced BST but nodes are intervals.

Define Node: Interval has l and r , while maxn is the **maximum r of all interval of the whole subtree**, and we use l as the key of BST.

```

struct interval {
    int l, r;
};

struct node {
    interval i;
    int maxn;
    node* left;
    node* right;
    node(interval in): i(in), maxn(in.r), left(NULL), right(NULL) {}
};

```

Insert: Like regular BST, using l as the key , and record maxn .

```

node* insert(node* u, interval i) {
    if(!u) return new node(i);
    if(i.l < u->i.l) u->left = insert(u->left, i);
    else u->right = insert(u->right, i);
    u->maxn = max(u->maxn, i.r);
    return u;
}

```

Search: To get all the intervals who are overlapping with i , there are 3 situations

1. If the node has overlap with i , record the answer.
2. If its left subtree's maxn is greater than $i.l$, search in the left subtree.
3. If the node's l is less than $i.r$, search in the right subtree.

```

bool isOverlap(interval a, interval b) {
    return a.l <= b.r && a.r >= b.l;
}

void searchAll(node* u, interval i, vector<interval>& re) {
    if(!u) return;
    if(isOverlap(u->i, i)) re.push_back(u->i);
    if(u->left && u->left->maxn >= i.l) searchAll(u->left, i, re);
    if(u->right && u->i.l <= i.r) searchAll(u->right, i, re);
}

```

The regular Interval Tree is like BST with worst time complexity $O(n)$ for each operation, but it can be optimized by AVL tree or other balanced tree, reducing the time complexity to $O(\log n)$.

vEB Tree

An vEB (**van Emde Boas**) Tree is a data structure that can support below functions in a integer set $S \subseteq \{0, 1, \dots, U-1\}$, while U is the range of integers (e.g. $U=2^{16}$)

L6 Hashing

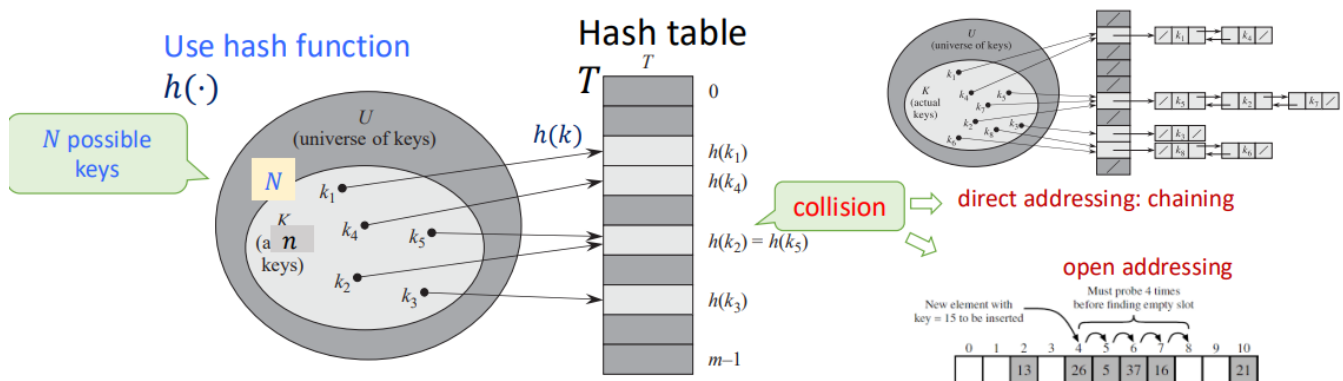
Basic Concepts of Hashing

Hash Function: A function that maps data of *arbitrary size* to *fixed-sized* values. And the goal is to propose a data structure such that for key k :

- $\text{add}(k), \text{delete}(k) = O(1)$
- $\text{search}(k) = O(1)$ on the average

From what I've learnt, the most important thing is not the precise hashing function itself, but the **critical thought of hashing**.

Hash Table: A table that stores indice and values of hashing.



Making Choices

Choosing the Keys:

- **By Nature:** Randomly from U . (Simple uniform hashing)
- **By an Opponent:** That knows the hash function we use when this is fixed, in a way to create the longest possible chain. (Universal hashing)

Choosing the Hash Functions:

- We use a **fixed** hash function everyone knows.
- We **change** before each experiments the hash function, chosen randomly from a "**universal set**" of functions known to everyone.

Special Case (perfect hashing): We know the keys, then choose the hash function.

Performance: Average length of chain in a *typical* slot (same hashing value).

Models for Randomness

Simple Uniform Hashing Assumption (SUHA)

h is fixed: In every experiment each key k 's value is chosen randomly (uniformly) from U .

- Then, **h is "good"** if $h(k)$ scatters uniformly over the hash table: $P_k[h(k) = i] = \frac{1}{m}$, $\forall i = 1, 2, \dots, m$

- We call this the **Simple Uniform Hashing Assumption**.

Universal Hashing

The set of keys to be stored is assumed fixed: In every experiment, h is randomly (uniformly) chosen from a set H .

- Then, **H is "good"** if $P_h[h(k_1) = h(k_2)] \leq \frac{1}{m}, \forall k_1 \neq k_2$
- H is a set of **Universal Hash Functions**

Actually, Universal Hashing is not a hash function, but designing a set of hash functions. It will randomly choose a hash function when using. Or we can say **Universal Hashing is using SUHA by randomization**, in order to decrease the probability of collisions.

Universal Hashing Function Examples

The Matrix Method

Settings:

- Keys are u -bits long, $|U| = 2^u$
- Hash values are b -bits long, hash table size $m = 2^b$

And we pick $h =$ random $b \times u$ $\{0/1\}$ matrix, define: $h(x) = h \cdot x \pmod 2$

Which means matrix multiplication under module 2.

Claim: For $x \neq y$, $P_h[h(x) = h(y)] \leq \frac{1}{m} = \frac{1}{2^b}$

e.g. $x = 2 = \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix}$, $h = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 \end{bmatrix}$ And we get $h(x) = h \cdot x = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix} = 6$

Linear Hashing for Vector Keys

Settings:

- Suppose keys are vectors $x = (x_1, x_2, x_3, x_4)$, where x_i is an 8-bit integer. (i.e. $0 \leq x \leq 255$)
- We use a hash table of size $m = p$, where p is a prime number slightly greater than 255 (e.g. 257).

Choose the vector of coefficients $a = (a_1, a_2, a_3, a_4)$ where a_i is (uniformly) randomly selected in the range $\{0, 1, \dots, m-1\}$ And we define: $h_a(x) = \left(\sum_{i=1}^4 a_i x_i \right) \pmod m$

So the hash family is $H = \{ h_a : a \in [0, m-1]^4 \}$

Claim: For $x \neq y$, $P_a[h_a(x) = h_a(y)] = \frac{1}{m}$

Affine Hashing for Integer Keys

Also called the *Carter-Wegman* construction

Settings:

- Keys are integers $k \in U$
- Pick a prime number p s.t. $|U| \leq p \leq 2|U|$
- Table size $m \leq p$

We randomly choose $a \in \{1, 2, \dots, p-1\}$, $b \in \{0, 1, \dots, p-1\}$, and define: $h_{a,b}(k) = ((a \cdot k + b) \bmod p) \bmod m$

Claim: For $x \neq y$, $P_{a,b}[h_{a,b}(x) = h_{a,b}(y)] \leq \frac{1}{m}$

e.g. $|U| = 2^4 = 16$, $p = 17$, $m = 6$, $k = 8$, $a = 3$, $b = 4$, then: $h_{3,4}(8) = ((3 \cdot 8 + 4) \bmod 17) \bmod 6 = 5$

Rolling Hash

Sliding window, but in hashing.

Rabin-Karp Algorithm

Consider a string $S = s_0 s_1 s_2 \dots s_{n-1}$. We define a window of length k s.t. $T = s_i s_{i+1} \dots s_{i+k-1}$. And we use the polynomial hash function: $H_i = H(T) = (s_i \cdot p^{k-1} + s_{i+1} \cdot p^{k-2} + \dots + s_{i+k-1} \cdot p^0) \bmod M$ Where p is the base, M is a large prime number and s_i is the ASCII number.

From the function above we can get: $H_{i+1} = ((H_i - s_i \cdot p^{k-1}) \cdot p + s_{i+k}) \bmod M$

And then we can $O(1)$ check whether a string of length k is appeared in S .

Table Doubling

We define **Load Factor**: $\alpha = \frac{n}{m}$ Where n is the element number in hash table, m is the bucket number of hash table.

We have known that we need a hash table to store the keys and values, but if the **element number in table is too large**, it will cause more and more collisions, which slowing down the hashing progress.

So if α is too large (usually 1), we will double the bucket number m , then rehash all the elements into new table.

Average Cost of Insertions $= \frac{2^{i+2}c + 2^i c_0}{2^i} = O(1)$

Or observe that up to any time t : by paying $O(1)$ per insert, covers at all times the total actual cost. We say that insert has an **amortized cost** of $O(1)$.

Perfect Hashing**Bloom Filters****L7 Amortized Analysis**

Why Amortized Complexity?

Think about doubling hash tables, some of the functions are *cheap*, like $O(1)$ calculate hashing, but some are *expensive*, like $O(n)$ to doubling hash tables.

Amortized analysis is a way to make sure each function is in an **average time**, no need to assume randomized input.

Actual Cost and Amortized Cost

$c(t)$ = **actual cost** of the operation at time t . It can be a function of the state of the data structure, e.g. depends on n = number of elements stored at time t .

$a(t)$ = **amortized cost** of the operation at time t . Assume $a(t) = O(1) \forall t$, i.e., $a(t) \leq c_a \forall t$

Suppose that we have shown that on any trajectory $0, 1, \dots, T$, $\sum_{0 \leq t \leq T} c(t) \leq \sum_{0 \leq t \leq T} a(t)$

Which means at any operation, the total actual cost $\leq$ the total budget cost we assigned. That is, our budget is enough to cover all the cost.

For any length T , $\sum_{0 \leq t \leq T} c(t) \leq \sum_{0 \leq t \leq T} a(t) \leq c_a T$

Amortization Methods

Aggregate Method

The easiest way of amortization.

$\text{amortized cost per op} = \frac{\text{total cost of } k \text{ ops}}{k}$

Amortized Bound

Define an amortized cost $a_{op}(t) = O(1)$ to each operation type s.t. for **any possible sequence of operations**, the total cost is covered. That is, $\sum(\text{amortized costs}) \geq \sum(\text{actual costs})$

Methods: **Accounting** method, **Charging** method, **Potential** method.

L8 Graphs

Graph Definition

$G = (V, E)$

V : A set of **vertices**. $|V|$ usually denoted by n .

$E \subseteq V \times V$: A set of **edges** (pair of vertices). $|E|$ usually denoted by m .

$m \leq n^2 = O(n^2)$

Two flavors:

- **Directed graph:** Edges have order.
- **Undirected graph:** Ignore order. Then only $\frac{n(n-1)}{2}$ possible edges.

Problems Relates to Graphs

Reachability: Find all states reachable from a given state.

Shortest Paths: Find the shortest path between state s_0 and s_d .

Cycle: Find if a directed graph has a cycle.

Topological Sort: Find a *causal ordering* of the nodes if it exists.

Minimum Spanning Tree: Find the *minimum* subgraph that contains all nodes.

Strongly Connected Components: Maximal sets of nodes that from every node can reach any other node in the set (mixing).

Graph Representation

Four representations with pros/cons:

- **Adjacency Lists** (of *neighbors* for each vertex)
- **Incidence Lists** (of *edges* from each vertex)
- **Adjacency Matrix** (0-1, which pairs are adjacent)
- Implicit representation (as a *find neighbor* function)

DFS

Depth-First Search (DFS) is a versatile linear-time procedure that **reveals a wealth of information** about a graph.

```
int n, m;
vector<int> g[MAXN];
int vis[MAXN]; // 0: WHITE, 1: GREY, 2: BLACK
int d[MAXN], f[MAXN], cnt;

void dfs(int u) {
    cnt++;
    d[u] = cnt;
    vis[u] = 1;
    for(int x = 0; x < g[u].size(); x++) {
        int v = g[u][x];
        if(!vis[v]) dfs(v);
    }
    cnt++;
    f[u] = cnt;
    vis[u] = 2;
}
```

Properties of DFS

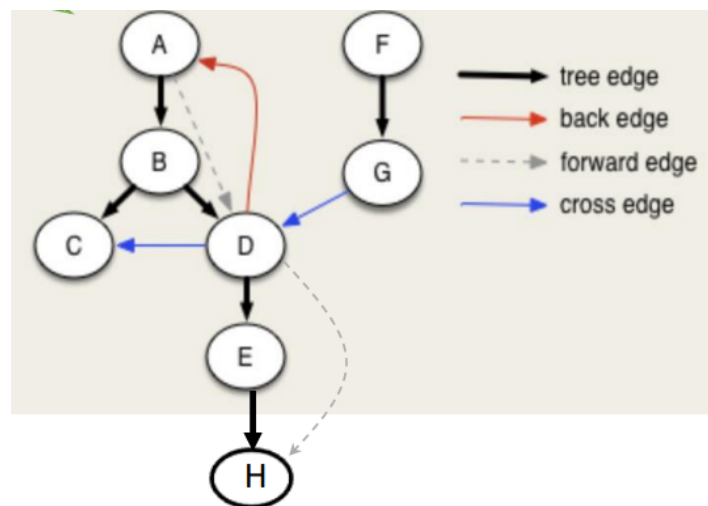
When DFS a graph, we will record two time stamp of each vertex:

- $d[u]$: **Discover Time**, when u is first visited.
- $f[u]$: **Finish Time**, when we finish exploring all neighbors of u .

Of course, it's easy to see that $u.d < u.f$

Then we can get 4 different type of edges, for a $u \rightarrow v$:

1. **Tree Edge**: When we **first discover** a new vertex v from u .
 - $u.d < v.d < v.f < u.f$
2. **Back Edge**: An edge that *points back to an ancestor* in the DFS tree.
 - $v.d < u.d < u.f < v.f$
3. **Forward Edge**: An edge that goes *from a node to its descendant* in the DFS tree, but **not** a direct tree edge.
 - $u.d < v.d < v.f < u.f$
4. **Cross Edge**: An edge that goes *between two different DFS branches (neither ancestor nor descendant)*.
 - $v.d < v.f < u.d < u.f$

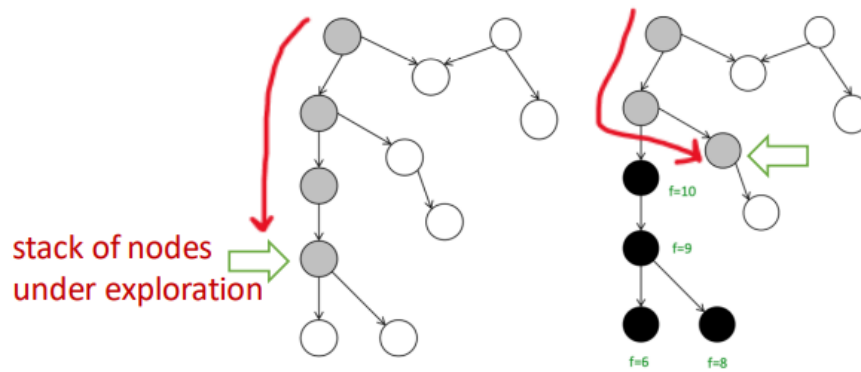


White Path Theorem

We colored the vertex in DFS:

- **White**: Not yet visited
- **Gray**: Currently being explored
- **Black**: Finished exploring

If, during a DFS, there is a path from a vertex u to a vertex v that only goes through *white* vertices, then **v will become a descendant of u** in the DFS tree.



We can use it to **detect cycle** in graph: When we see a **back edge**, or **meet non-white node**, means we found a path from a node u back to one of its ancestors v , and there is a cycle.

Topological Sort

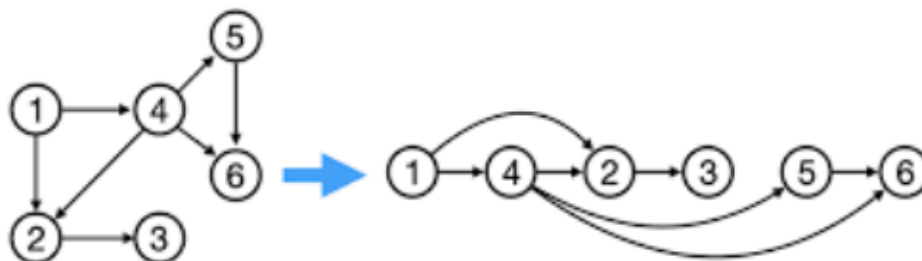
Topological Sort of a DAG $G=(V,E)$.

Theorem: G is a DAG iff there is a **linear ordering** of all its $v \in V$ s.t., if $(u, v) \in E$, then $u < v$ in the above ordering. Or we say (u,v) goes always from left to right.

Algorithm: Given DAG G .

1. Run any DFS(G), compute finishing time of nodes.
2. Linear order: The nodes in decreasing order of finishing times.

Corollary: In a DAG, $(u, v) \in E$ iff $u.f > v.f$ for any DFS.



BFS

SCC